

WORKSHOP

Programming Patterns & Problem Solving in JavaScript

Think first. Then code. Six patterns built from nothing but loops and conditions.

Instructor — Dilovan Matini



CENTER FOR ACADEMIC AND PROFESSIONAL ADVANCEMENT
THE AMERICAN UNIVERSITY OF KURDISTAN

Five things you will be able to do by the end

Learning objectives — this workshop trains problem-solving habits first; the syntax follows.

01 **Think before writing code**

Plan the steps on paper before you touch the keyboard.

02 **Break problems into logical steps**

Turn one big question into small, ordered decisions a computer can follow.

03 **Solve problems with loops and conditions**

The two tools hiding behind every pattern we will build today.

04 **Build common JavaScript methods manually**

Recreate find, filter, map, and sort from scratch — then the built-ins make sense.

05 **Grow real problem-solving skill**

Patterns you can reuse in any language, any framework, any interview.

An algorithm is just a sequence of steps

Warm-up: "How do you make a cup of tea?" — you already think in algorithms.

THE IDEA

An **algorithm** = a sequence of steps that solves a problem. The steps must be **ordered**, **unambiguous**, and **finite** — a computer can follow them without guessing.

WARM-UP — MAKE A CUP OF TEA

- 1 Boil water
- 2 Put a tea bag in the cup
- 3 Pour the water, wait 3 minutes
- 4 Remove the tea bag
- 5 Drink

Change the order — or skip a step — and the recipe breaks. Same for code.

● TODAY'S CHALLENGE

How would you tell a computer to find the largest number in a list — step by step?

A computer cannot "just look" at the list. You must give it an algorithm: start somewhere, compare, remember, repeat.

Today's focus: **problem-solving** — not syntax memorization. If you can say the steps, you can write the code.

PATTERN 1 / 6

Linear search — check each item, stop when found

Problem: find "React" inside an array of books — without using `.find()` or `.includes()`.

HOW IT WORKS

- 1 Start at the first item
- 2 Compare it with the target
- 3 Match? Stop — **break**
- 4 No match? Move to the next
- 5 End of the list = not found

This is exactly what `.find()` does for you — one item at a time, in order.

```
const books = ["HTML", "CSS", "React", "Node.js"];
```

```
const target = "React";
```

```
for (let i = 0; i < books.length; i++) {  
  if (books[i] === target) {  
    console.log("Found:", books[i]);  
    break; // stop searching once found  
  }  
}
```

- **Try it:** search for "CSS", "PHP", and "Node.js". What changes each time — the loop, or only the target? What prints when the book is not in the list?

PATTERN 2 / 6

Maximum value — remember the best so far

Problem: find the largest number in an array without using `Math.max()`.

HOW IT WORKS

- 1 Assume the first number wins
- 2 Compare the next number with it
- 3 Bigger? Replace the champion
- 4 Repeat to the end of the list
- 5 What remains is the largest

One running variable — **largest** — is the whole trick. This is how `.sort()` and `Math.max()` think too.

```
const numbers = [12, 45, 7, 89, 23];
let largest = numbers[0]; // assume the first wins

for (let i = 1; i < numbers.length; i++) {
  if (numbers[i] > largest) {
    largest = numbers[i]; // new champion
  }
}
```

Trace **largest**: 12 → **45** → **89** — the 7 and the 23 change nothing.

-
- **Try it:** flip one character to find the **smallest** number instead. Then use the same pattern on strings — find the **longest word** in a list of book titles.

PATTERN 3 / 6

Counting — loop, test, and add one

Problem: count how many even numbers are in an array.

HOW IT WORKS

- 1 Start a counter at zero
- 2 Take each number in turn
- 3 Test: $n \% 2 === 0$?
- 4 Yes? Add one to the counter
- 5 The counter is the answer

`%` gives the remainder of a division. Even numbers leave 0 when divided by 2 — odd numbers leave 1.

```
const numbers = [3, 8, 12, 5, 20, 7];
let count = 0; // start from zero

for (const n of numbers) {
  if (n % 2 === 0) {
    count++; // one more even number
  }
} // count === 3
```

Same skeleton as before: one loop, one test, one running variable — `count` instead of `largest`.

-
- **Try it:** change the test to count `odd` numbers. Then count how many book titles are `longer than 5 letters` — which property replaces `n % 2`?

Filter and map — build the new array yourself

Same loop skeleton, different job — filter selects, map transforms.

PATTERN 4 · FILTER — KEEP SOME ITEMS

```
const nums = [1, 2, 3, 4, 5, 6];
const evens = []; // the new list

for (const n of nums) {
  if (n % 2 === 0) {
    evens.push(n); // keep it
  }
}

// same as nums.filter(n => n % 2 === 0)
```

PATTERN 5 · MAP — TRANSFORM EVERY ITEM

```
const names = ["html", "css", "react"];
const upper = []; // the new list

for (const name of names) {
  upper.push(name.toUpperCase());

  // transform, then keep
}

// same as names.map(n => n.toUpperCase())
```

-
- **Now you understand what `.filter()` and `.map()` do internally** — they are not magic. They are loops someone else already wrote, with your test or transform plugged in.

PATTERN 6 / 6

Bubble sort — swap neighbours until sorted

How does JavaScript sort numbers? One answer — compare neighbours, swap, repeat.

TWO LOOPS, TWO JOBS

Outer loop = passes. Each pass pushes the largest remaining value to the end.

Inner loop = comparisons. Compare each pair of neighbours along the way.

Swap with a temp variable when the left one is bigger.

WATCH ONE PASS

Start [5, 3, 8, 1]

Pass 1 [3, 5, 1, 8] — 8 bubbles up

Pass 2 [3, 1, 5, 8]

Pass 3 [1, 3, 5, 8] — sorted

```
const arr = [5, 3, 8, 1];

for (let i = 0; i < arr.length; i++) {
  // one full pass per i
  for (let j = 0; j < arr.length - 1; j++) {
    if (arr[j] > arr[j + 1]) {
      const temp = arr[j];
      arr[j] = arr[j + 1];
      arr[j + 1] = temp; // swap
    }
  }
}
```

- **Discuss:** why does the inner loop stop at `arr.length - 1`? And after pass 1, which position is guaranteed to be final?

Final challenge — five problems, no built-in methods

Individual work. Loops and conditions only — no `.filter()`, `.includes()`, `Math.max()`, or `.reverse()`.

01 Find the second largest number

Hint — track two variables: the largest and the runner-up.

02 Remove duplicate numbers

Hint — build a new list; add a number only if it is not already inside (search it yourself).

03 Find the longest word

Hint — same champion pattern as the maximum value, but compare word lengths.

04 Count the vowels in a string

Hint — loop over the characters; count each one that is a, e, i, o, or u.

05 Check if a word is a palindrome

Hint — compare the first and last letters, then move inward. "level" passes, "hello" fails.

THE BIG IDEA

find(), filter(), map(), sort() — they are just algorithms someone else already wrote.

Today was never about JavaScript syntax — it was about how programmers think.

DISCUSS

Which pattern felt the easiest? Which one was the hardest? Which built-in method makes more sense now that you built it by hand?